Paper — Progetto Finale Cybersecurity

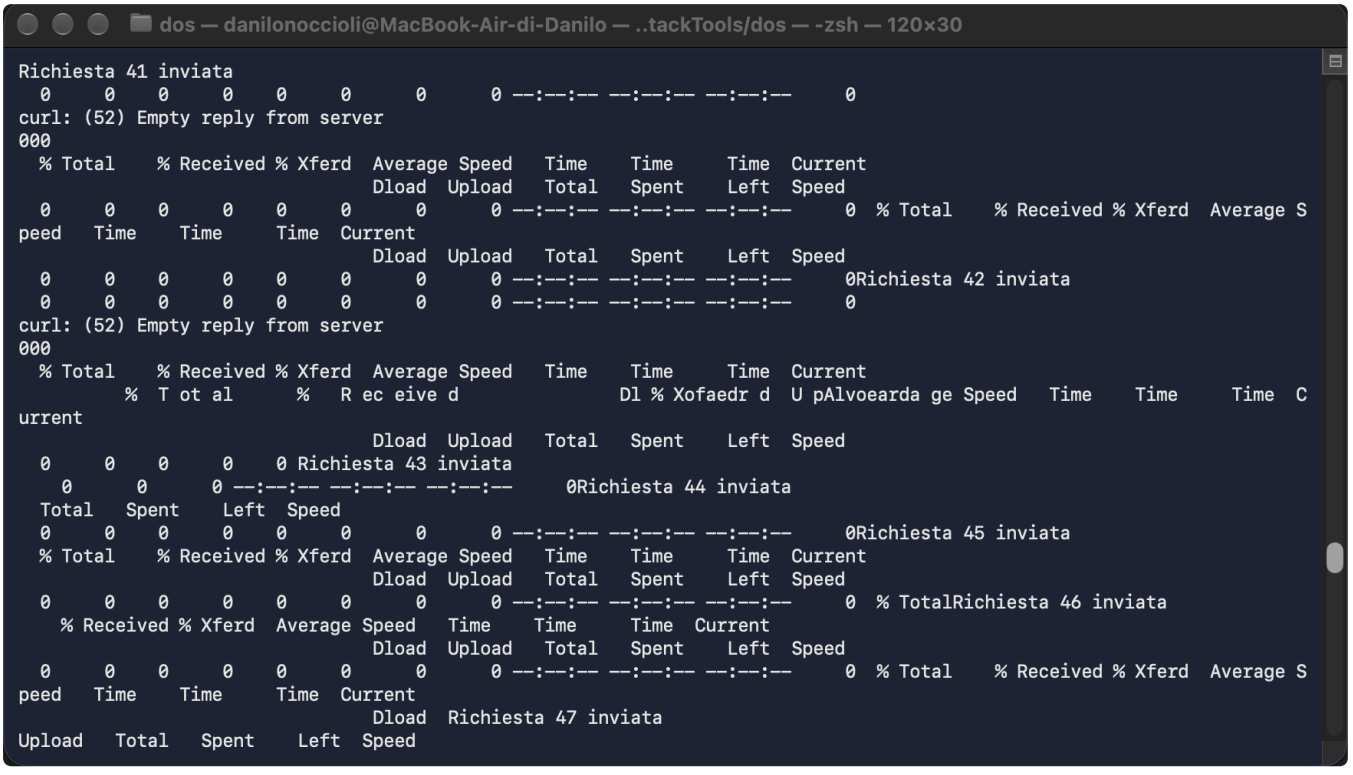
Analisi di sicurezza del progetto finale (Aulab Hackademy, specializzazione Cybersecurity) — Danilo Noccioli

Challenge 1 — Rate Limiter Mancante

Scenario / Vulnerabilita'

La rotta pubblica di ricerca articoli (`/articles/search`) non aveva alcun limite sul numero di richieste accettate. Un utente non autenticato puo' quindi inviare un volume arbitrario di richieste in poco tempo, saturando il server.

Lanciando le 100 richieste in parallelo contro il server di sviluppo (`php artisan serve`), dopo circa 45-50 richieste il server smette di rispondere: le curl restituiscono `curl: (52) Empty reply from server` e codice `000` , anche dopo la fine dello script. Il log del server mostra inoltre `Invalid request (Malformed HTTP request)` per decine di connessioni simultanee, segno che il server di sviluppo (single-threaded) va in saturazione.



Screenshot del terminale durante l'esecuzione dell'attacco DoS su /articles/search: il server PHP non riesce piu a rispondere e le richieste falliscono con errore curl 52 (Empty reply from server).

Sono stati applicati tre livelli, dal piu' generico al piu' mirato:

Livello 1 — throttle sulla rotta pubblica (`routes/web.php`):

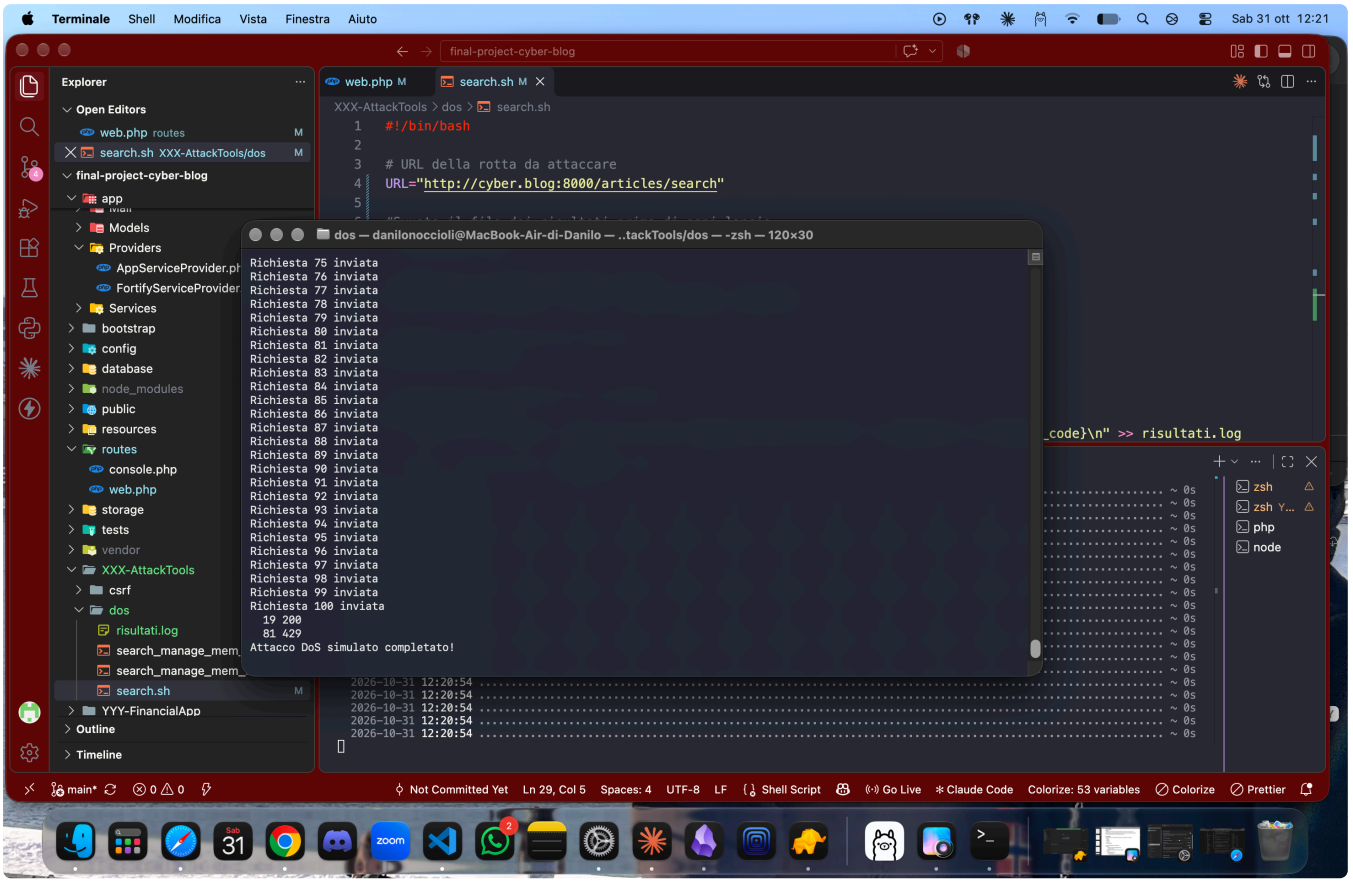
`throttle:20,1` limita a 20 richieste al minuto per IP, rispondendo 429 oltre il limite. Il valore (20-30/min) e' stato scelto come compromesso tra usabilita' per una ricerca pubblica e protezione da abusi.

Livello 2 — blocco temporaneo per IP con log

Il middleware tiene un contatore in cache per ogni IP (chiave offuscata con sha1): oltre 5 richieste in un minuto, l'IP viene marcato come bloccato per 1 minuto, con redirect alla homepage e scrittura di un `Log::warning`. Registrato come alias in `bootstrap/app.php`:

Livello 3 — rate limit sull'autenticazione. Gia' presente nel progetto grazie a Fortify (`app/Providers/FortifyServiceProvider.php`): `RateLimiter::for('login', ...)` limita a 5 tentativi al minuto, con chiave basata su email+IP, a protezione dal brute force sulle credenziali.

Prima di aggiungere il middleware `block.suspicious` (Livello 2), un primo test con solo il throttle di Laravel (Livello 1) attivo aveva dato questo risultato:



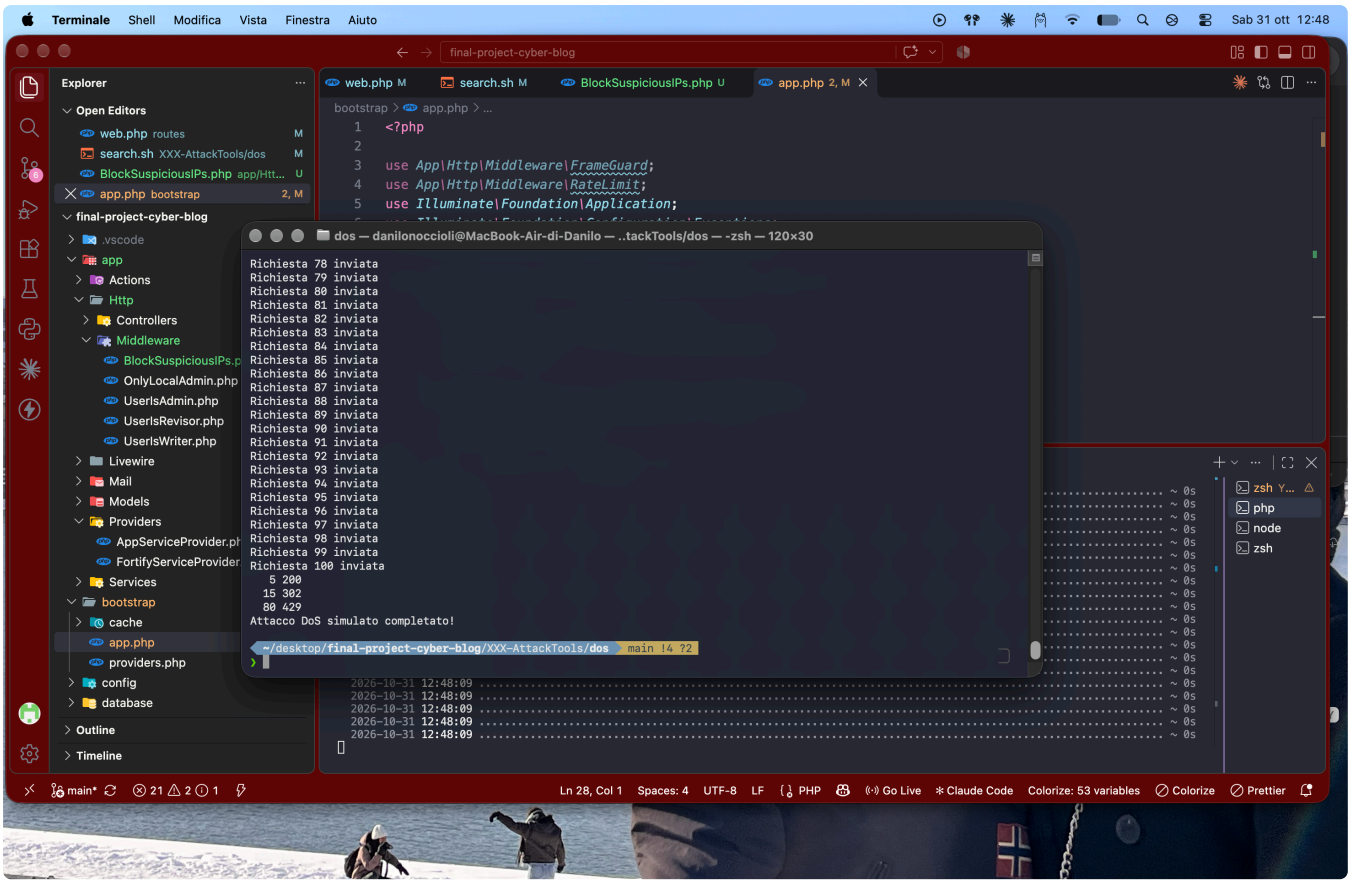
Con il solo throttle, le richieste in eccesso tornano subito 429 (limite 20/min) senza passare dal blocco IP: infatti non compare nessun 302.

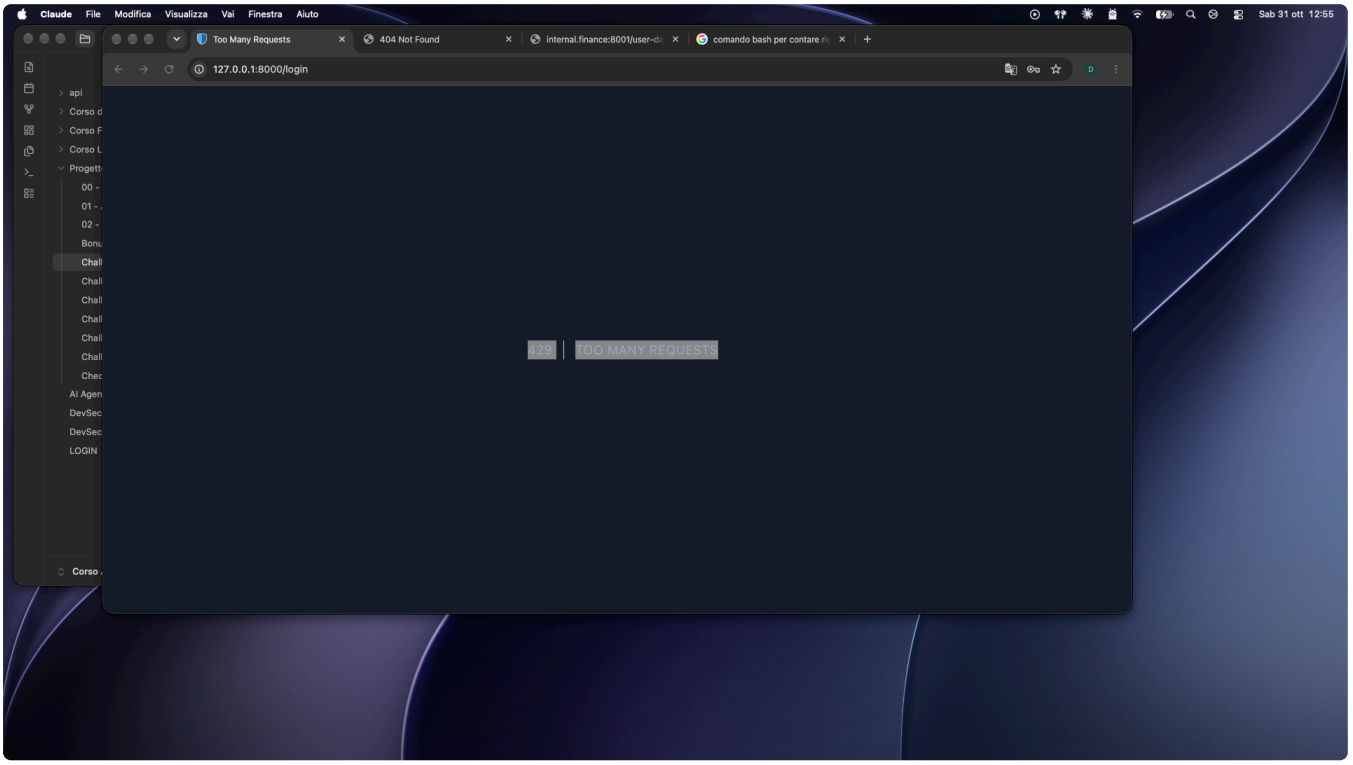
Dopo aver aggiunto anche il middleware di blocco IP (Livello 2), il test finale e' il seguente:

Rilanciando `search.sh` (sequenziale, senza `&`, per evitare falsi errori di connessione dovuti al server single-threaded) dopo aver applicato i livelli 1 e 2:

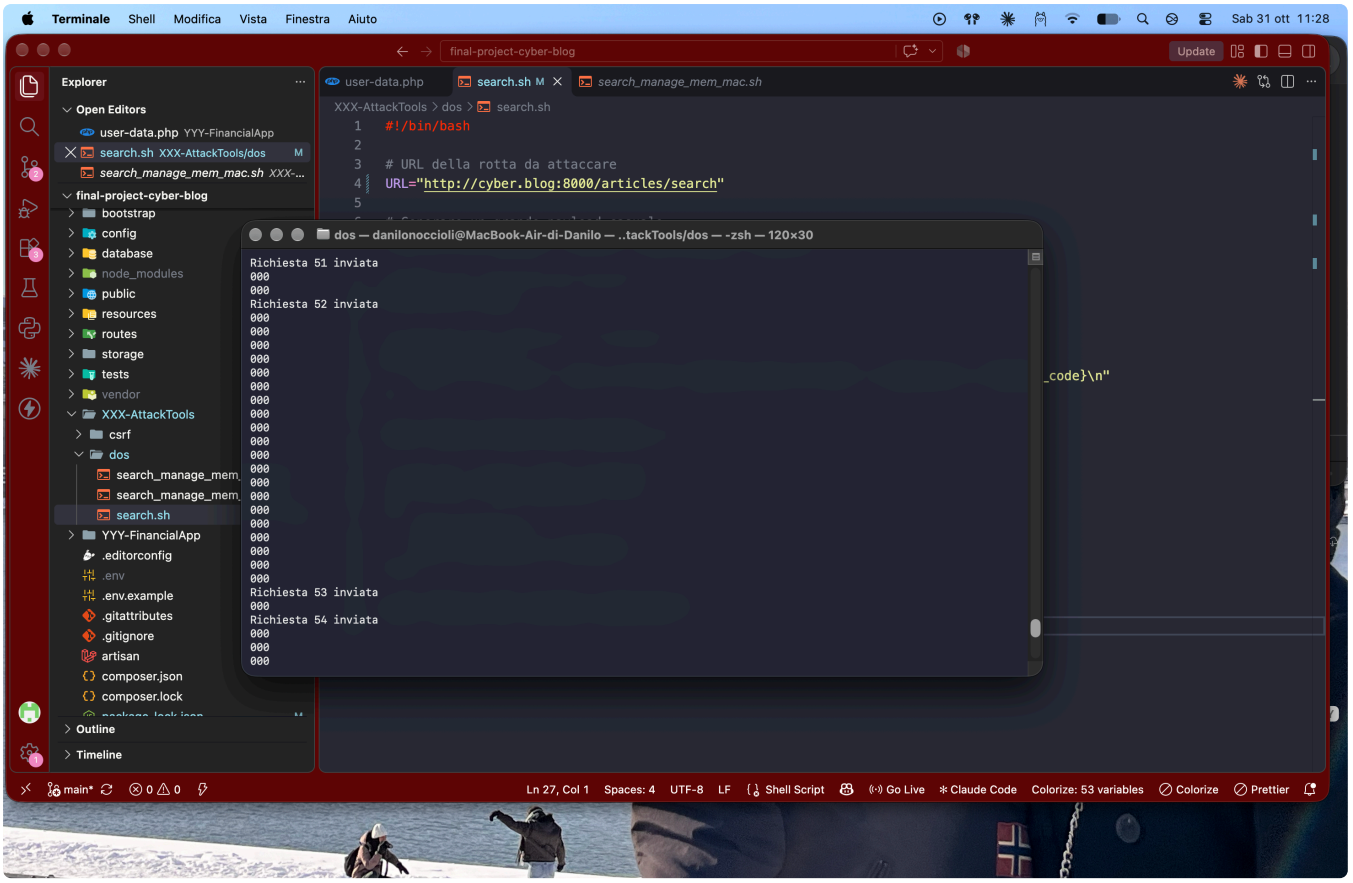
Le prime 5 richieste passano (200), le successive 15 vengono reindirizzate (302, bloccate da `block.suspicious`), le restanti 80 ricevono 429 (throttle). Il server resta reattivo per tutta la durata del test, a differenza del crash osservato prima della mitigazione.

Verifica del login: dopo 5 tentativi di accesso con password errata, il sistema risponde `429 Too Many Requests`.





Verifica del rate limiter sul login di Fortify: dopo 5 tentativi con password errata, il browser mostra 429 Too Many Requests.



Bonus

Bonus 1 — Rate limiter sul login di Fortify

Gia' soddisfatto dal Livello 3 di mitigazione della Challenge 1: il `RateLimiter::for('login', ...)` presente in `app/Providers/FortifyServiceProvider.php` limita i tentativi di login a 5 al minuto per combinazione email+IP, prevenendo il brute force sulle credenziali (collegato ad A07 - Identification and Authentication Failures).

Hardening aggiuntivo delle sessioni (iniziativa personale)

Oltre a quanto richiesto, ho rafforzato la configurazione delle sessioni in `.env`, sempre nell'ambito di A07 (Broken Authentication / gestione sessioni):

- `SESSION_DRIVER=database`: le sessioni sono salvate nel database invece che su file, permettendo di invalidarle/revocarle lato server (ad esempio in caso di compromissione di un account).

- `SESSION_ENCRYPT=true`: i dati di sessione vengono cifrati, riducendo il rischio di lettura o manomissione se qualcuno accede allo storage delle sessioni.
- `SESSION_LIFETIME=120`: la sessione scade dopo 120 minuti di inattivita', limitando la finestra di utilizzo di una sessione rubata o lasciata aperta.
- `SESSION_EXPIRE_ON_CLOSE=true`: la sessione termina alla chiusura del browser, riducendo il rischio di sessioni abbandonate su dispositivi condivisi.